基于角色权限控制的校园数字证书联盟链存证系统

1. 唤醒区块链环境

启动 4 节点底层链:

cd ~/fisco

bash nodes/127.0.0.1/start_all.sh

```
cyx@cyx-virtual-machine:~$ cd ~/fisco
cyx@cyx-virtual-machine:~/fisco$ bash nodes/127.0.0.1/start_all.sh
try to start node0
try to start node1
try to start node2
try to start node3
 node2 start successfully
 node1 start successfully
 node0 start successfully
 node3 start successfully
```

进入区块链控制台:

cd console

bash start.sh

```
cyx@cyx-virtual-machine:~/fisco$ cd console
cyx@cyx-virtual-machine:~/fisco/console$ bash start.sh
=====================================================================================================
Welcome to FISCO BCOS console(2.10.0)!
Type 'help' or 'h' for help. Type 'quit' or 'q' to quit console.
 _______ ______  _______ ______  _______ ______       _______ ______  _______ ______  _______ ______
|       |      \/       \      \/       \      \     |       \      \/       \      \/       \      \
| $$$$$$$\$$$$$|  $$$$$$|  $$$$$$|  $$$$$$\    | $$$$$$$|  $$$$$$|  $$$$$$|  $$$$$$|  $$$$$$\
| $$__     | $$ | $$___\$| $$   \$| $$  | $$    | $$__/ $| $$   \$| $$  | $| $$___\$$
| $$  \    | $$  \$$    \| $$     | $$  | $$    | $$    $| $$     | $$  | $$\$$    \
| $$$$$    | $$  _\$$$$$$| $$   __| $$  | $$    | $$$$$$$| $$   __| $$  | $$_\$$$$$$\
| $$      _| $$_|  \__| $| $$__/  | $$__/ $$    | $$__/ $| $$__/  | $$__/ $|  \__| $$
| $$     |   $$ \\$$    $$\$$    $$\$$    $$    | $$    $$\$$    $$\$$    $$\$$    $$
 \$$      \$$$$$$ \$$$$$$  \$$$$$$  \$$$$$$      \$$$$$$$  \$$$$$$  \$$$$$$  \$$$$$$

=====================================================================================================
[group:1]>
```

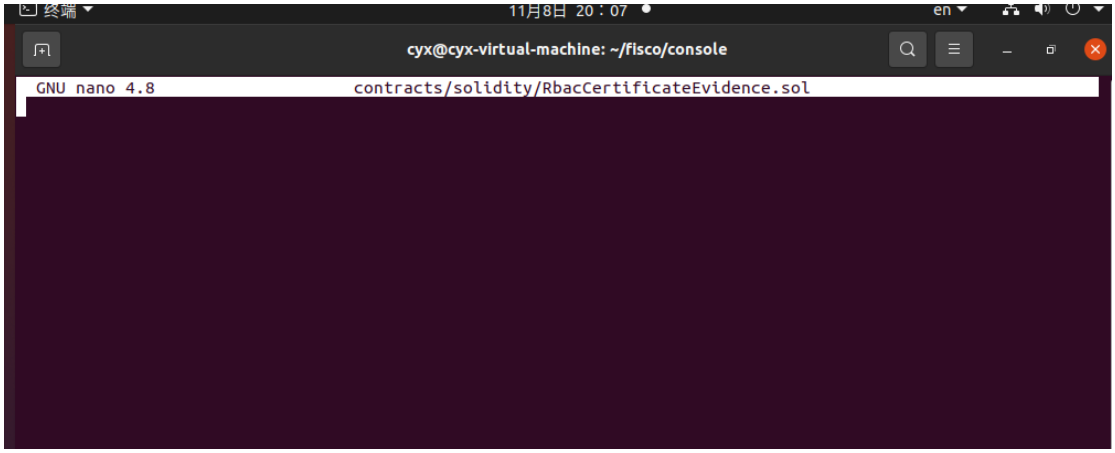
2. 编写与部署 V2.0 合约

退出控制台以编辑文件:

在 [group1]: 后面输入小写字母 q 并回车，退回普通终端界面。

新建 V2.0 权限合约文件:

nano contracts/solidity/RbacCertificateEvidence.sol



粘贴合约代码并保存：

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.4.25;


contract RbacCertificateEvidence {

    address public owner; // 教务处

    mapping(address => bool) public authorizedIssuers; // 授权名单


    struct Evidence {

        address issuer;

        uint256 timestamp;

        bool exists;

    }


    mapping(bytes32 => Evidence) private evidences;


    constructor() public {

        owner = msg.sender;

    }
```

```
    modifier onlyOwner() {
        require(msg.sender == owner, "Only Owner can do this");
        _;
    }

    modifier onlyAuthorized() {
        require(authorizedIssuers[msg.sender] || msg.sender == owner, "Not
authorized to issue");
        _;
    }

    function authorizeIssuer(address _issuer) public onlyOwner {
        authorizedIssuers[_issuer] = true;
    }

    function revokeIssuer(address _issuer) public onlyOwner {
        authorizedIssuers[_issuer] = false;
    }

    function createEvidence(bytes32 hash) public onlyAuthorized {
        require(!evidences[hash].exists, "This evidence already exists!");
        evidences[hash] = Evidence(msg.sender, block.timestamp, true);
    }

    function verifyEvidence(bytes32 hash) public view returns (address,
uint256, bool) {
        Evidence memory e = evidences[hash];
```

```
        return (e.issuer, e.timestamp, e.exists);
    }
}
```

重新进入控制台并部署合约：

bash start.sh

```
cyx@cyx-virtual-machine:~/fisco/console$ bash start.sh
=====================================================================================================
Welcome to FISCO BCOS console(2.10.0)!
Type 'help' or 'h' for help. Type 'quit' or 'q' to quit console.
 ________ ______  ______  ______  ______      _______  ______  ______  ______
|        |      \/      \ /      \ /      \    |       \ /      \ /      \ /      \
| $$$$$$$$\$$$$$|  $$$$$$|  $$$$$$|  $$$$$$\   | $$$$$$$|  $$$$$$|  $$$$$$|  $$$$$$\
| $$__     | $$ | $$___\$| $$   \$| $$  | $$   | $$__/ $| $$   \$| $$  | $| $$___\$$
| $$  \    | $$  \$$    \| $$     | $$  | $$   | $$    $| $$     | $$  | $$\$$    \
| $$$$$    | $$  _\$$$$$$| $$   __| $$  | $$   | $$$$$$$| $$   __| $$  | $$_\$$$$$$\
| $$      _| $$_|  \__| $| $$__/  | $$__/ $$   | $$__/ $| $$__/  | $$__/ $|  \__| $$
| $$     |   $$ \\$$    $$\$$    $$\$$    $$   | $$    $$\$$    $$\$$    $$\$$    $$
 \$$      \$$$$$$ \$$$$$$  \$$$$$$  \$$$$$$     \$$$$$$$  \$$$$$$  \$$$$$$  \$$$$$$

=====================================================================================================
[group:1]>
```

进入 [group1]: 后，输入：

deploy RbacCertificateEvidence.sol

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0xfdc01f8f936e2f6b1a90df246a65ec7e4b15597b361d5d4c0856ee700e21dbb8
contract address: 0x08e17124a09178cec68450e26712abfe1385677d
currentAccount: 0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69
```

获取“学生账户”地址：

listAccount

```
[group:1]> listAccount
0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69(current account) <=
```

可见这里只有一个默认账户，需新建一个“学生”账户来做测试：

newAccount

```
[group:1]> newAccount
AccountPath: account/ecdsa/0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143.pem
Note: This operation does not create an account in the blockchain, but only creates a local account, and deploying a contract through this account will create an account in the blockchain
newAccount: 0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143
AccountType: ecdsa
```

切换到学生账户：

loadAccount account/ecdsa/0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143.pem

```
[group:1]> loadAccount account/ecdsa/0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143.pem
Load account account/ecdsa/0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143.pem success!
```

越权拦截测试：

call RbacCertificateEvidence 0x08e17124a09178cec68450e26712abfe1385677d createEvidence 0xbbbb1111111111111111111111111111111111111111111111111111111111111111

```
[group:1]> call RbacCertificateEvidence 0x08e17124a09178cec68450e26712abfe1385677d createEvidence 0xbbbb11111
1111111111111111111111111111111111111111111111111111111111111111
 cannot encode in encodeMethodFromString with appropriate interface ABI, make sure params match
```

原因：ABI 缓存的报错！这说明控制台在你加载新账户的时候，把刚才部署的合约记忆给丢了。

重新部署：

loadAccount account/ecdsa/0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69.pem

deploy RbacCertificateEvidence.sol

```
[group:1]> loadAccount account/ecdsa/0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69.pem
Load account account/ecdsa/0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69.pem success!
```

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0xfef4ccab5e1d297b2ac499dc77fe62abff958d027d74e07312fa38bacf36ab1d
contract address: 0x6992f2af751bbd07cb7c070e8746ae9b6eda5aa5
currentAccount: 0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69
```

切回学生，测试越权拦截：

loadAccount account/ecdsa/0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143.pem

```
[group:1]> loadAccount account/ecdsa/0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143.pem
Load account account/ecdsa/0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143.pem success!
```

学生尝试发假证：

call RbacCertificateEvidence 0x6992f2af751bbd07cb7c070e8746ae9b6eda5aa5 createEvidence 0xbbbb1111111111111111111111111111111111111111111111111111111111111111

原因：CO BCOS 控制台的“隐藏坑”（这也是个非常好的技术经验）。

在较老的控制台版本中，每次切换账户（loadAccount），控制台就会清空内存里的合约 ABI 缓存。它失忆了，所以不知道 createEvidence 怎么编码。

控制台恢复“记忆”：

deploy RbacCertificateEvidence.sol

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0x760b12ee551a7bdaef36910ee1b11a75284fb6dfea6278a0b0c25719eb3c0114
contract address: 0x7db01bebb38e3e1d5cea2824e43d0a4dcc89e922
currentAccount: 0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143
```

测试越权拦截：

call RbacCertificateEvidence 0x6992f2af751bbd07cb7c070e8746ae9b6eda5aa5 createEvidence 0xbbbb1111111111111111111111111111111111111111111111111111111111111111

```
[group:1]> call RbacCertificateEvidence 0x6992f2af751bbd07cb7c070e8746ae9b6eda5aa5 createEvidence 0xbbbb11111
1111111111111111111111111111111111111111111111111111111111111111
 cannot encode in encodeMethodFromString with appropriate interface ABI, make sure params match
```

依旧不行，重置环境，获得干净的初始状态：

bash start.sh

建立“教务处”和“学生”两个角色：

listAccount

```
[group:1]> listAccount
0x8d1e4d4f79d5bf5a00e332823d2b73e70c2cc143(current account) <=
0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69
```

newAccount

```
[group:1]> newAccount
AccountPath: account/ecdsa/0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24.pem
Note: This operation does not create an account in the blockchain, but only creates a local account, and depl
oying a contract through this account will create an account in the blockchain
newAccount: 0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24
AccountType: ecdsa
```

测试：

1. **切换为教务处**

loadAccount account/ecdsa/0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69.pem

```
[group:1]> loadAccount account/ecdsa/0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69.pem
Load account account/ecdsa/0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69.pem success!
```

2. **教务处在区块链上部署主合约**

deploy RbacCertificateEvidence.sol

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0x2dba4955b702454a352a372048f5debd99592e362a6a228a20ba5a45803bf5b2
contract address: 0xeffca7447b3a18362f4df9a5fce27533a06dd583
currentAccount: 0x5bff9e15fdaee1c79058f41dbc2238b1eeb38b69
```

2.为学生账户

loadAccount account/ecdsa/0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24.pem

```
[group:1]> loadAccount account/ecdsa/0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24.pem
Load account account/ecdsa/0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24.pem success!
```

唤醒控制台记忆：

deploy RbacCertificateEvidence.sol

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0x27d830fda6ae7bef74e3fcf35ada2dfc6bb12c1deaac48e7ad2551b5a0b934a8
contract address: 0xe2b05d88cb35e0707066c4e19f7e98a85a173215
currentAccount: 0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24
```

学生尝试发假证（攻击教务处的主合约）：

call RbacCertificateEvidence 0xeffca7447b3a18362f4df9a5fce27533a06dd583 createEvidence xbbbb1111111111111111111111111111111111111111111111111111111111111111

```
[group:1]> call RbacCertificateEvidence 0xeffca7447b3a18362f4df9a5fce27533a06dd583 createEvidence 0xbbbb11111
1111111111111111111111111111111111111111111111111111111111111111
 cannot encode in encodeMethodFromString with appropriate interface ABI, make sure params match
```

原因：not encode in encodeMethodFromString 是 FISCO BCOS 2.10 控制台一个非常恶心的底层 Bug：它不支持在切换账户后，去调用其他账户部署的合约（因为它把 ABI 记忆清空了，且无法重新映射）

退出控制台，临时改代码

nano contracts/solidity/RbacCertificateEvidence.sol

找到合约第 17 行左右：constructor() public {

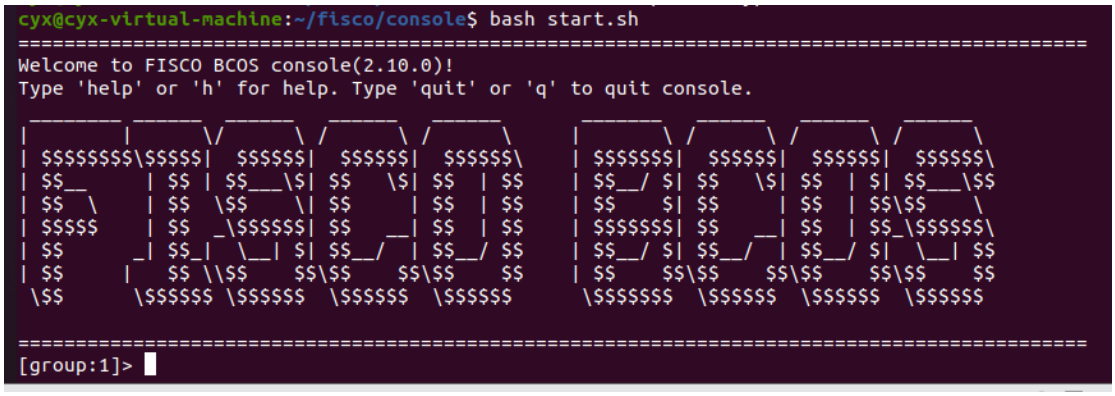
```
    owner = msg.sender;  //  把这一行改掉
}
```

把它改成：

```
constructor() public {
    owner = address(0);  // 故意把校长设为一个空地址，测试拦截用
}
```

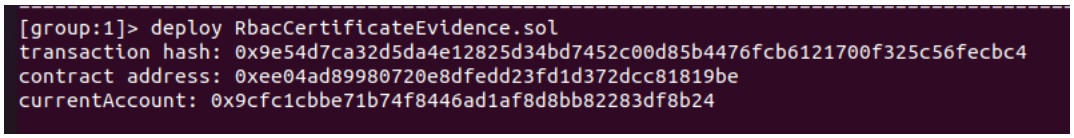
重新进入控制台：

bash start.sh



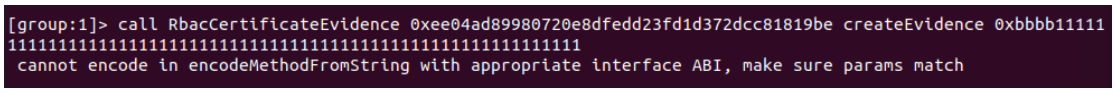
直接部署：

deploy RbacCertificateEvidence.sol



测试拦截：

call RbacCertificateEvidence 0xee04ad89980720e8dfedd23fd1d372dcc81819be createEvidence 0xbbbb11111111111111111111111111111111111111111111111111111111111111



依旧报错，选择并覆盖合约文件：

打开合约文件：

nano contracts/solidity/RbacCertificateEvidence.sol

替换为：

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.4.25;


contract RbacCertificateEvidence {

    address public owner;

    mapping(address => bool) public authorizedIssuers;
```

```
    struct Evidence {
        address issuer;
        uint256 timestamp;
        bool exists;
    }

    // 内部依然使用 bytes32 存储哈希
    mapping(bytes32 => Evidence) private evidences;

    constructor() public {
        // 正常部署，部署者即为教务处 (Owner)
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "Only Owner can do this");
        _;
    }

    modifier onlyAuthorized() {
        require(authorizedIssuers[msg.sender] || msg.sender == owner, "Not
authorized to issue");
        _;
    }

    function authorizeIssuer(address _issuer) public onlyOwner {
```

```
        authorizedIssuers[_issuer] = true;
    }

    function revokeIssuer(address _issuer) public onlyOwner {
        authorizedIssuers[_issuer] = false;
    }

    // 【核心修改点】：参数改为 string，内部转换为 bytes32
    function createEvidence(string hash) public onlyAuthorized {
        bytes32 hashKey = keccak256(abi.encodePacked(hash));
        require(!evidences[hashKey].exists, "This evidence already exists!");
        evidences[hashKey] = Evidence(msg.sender, block.timestamp, true);
    }

    // 【核心修改点】：参数改为 string，内部转换为 bytes32
    function verifyEvidence(string hash) public view returns (address,
uint256, bool) {
        bytes32 hashKey = keccak256(abi.encodePacked(hash));
        Evidence memory e = evidences[hashKey];
        return (e.issuer, e.timestamp, e.exists);
    }
}
```

重新部署合约：

bash start.sh

deploy RbacCertificateEvidence.sol

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0x2039e4cba1bd653d3410fde407d805f795d793d2d9eae00b5ae1a5a1c2cdcba5
contract address: 0xe56915632c7ec33b092e72b4749f334da3bef6f8
currentAccount: 0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24
```

测试：

校长账户发证：

call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 createEvidence "0xaaaa1111111111111111111111111111111111111111111111111111111111111111"

```
[group:1]> call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 createEvidence "0xaaaa1111
1111111111111111111111111111111111111111111111111111111111111111"
transaction hash: 0x190d41927a7a8a63bd7ad2c9a3fa9e5ad3a2bca274b056197f42066e2b0a4b36
---------------------------------------------------------------------------------------
transaction status: 0x0
description: transaction executed successfully
---------------------------------------------------------------------------------------
Transaction inputs:
Input value size:1
Input types: (STRING)
Input values:(0xaaaa1111111111111111111111111111111111111111111111111111111111111111)
---------------------------------------------------------------------------------------
Receipt message: Success
Return message: Success
Return values:[]
---------------------------------------------------------------------------------------
Event logs
Event: {}
```

创建一个无权限账户：

```
[group:1]> newAccount
AccountPath: account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem
Note: This operation does not create an account in the blockchain, but only creates a local account, and depl
oying a contract through this account will create an account in the blockchain
newAccount: 0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f
AccountType: ecdsa
```

切换为无权限账户：

loadAccount account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem

```
[group:1]> loadAccount account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem
Load account account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem success!
```

唤醒控制台记忆：

deploy RbacCertificateEvidence.sol

尝试伪造证书上链：

call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 createEvidence "0xbbbb1111111111111111111111111111111111111111111111111111111111111111"

```
[group:1]> call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 createEvidence "0xaaaa1111
1111111111111111111111111111111111111111111111111111111111111111"
transaction hash: 0x190d41927a7a8a63bd7ad2c9a3fa9e5ad3a2bca274b056197f42066e2b0a4b36
---------------------------------------------------------------------------------------
transaction status: 0x0
description: transaction executed successfully
---------------------------------------------------------------------------------------
Transaction inputs:
Input value size:1
Input types: (STRING)
Input values:(0xaaaa11111111111111111111111111111111111111111111111111111111111111111)
---------------------------------------------------------------------------------------
Receipt message: Success
Return message: Success
Return values:[]
---------------------------------------------------------------------------------------
Event logs
Event: {}

[group:1]> newAccount
AccountPath: account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem
Note: This operation does not create an account in the blockchain, but only creates a local account, and depl
oying a contract through this account will create an account in the blockchain
newAccount: 0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f
AccountType: ecdsa

[group:1]> loadAccount account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem
Load account account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem success!

[group:1]> call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 createEvidence "0xbbbb1111
1111111111111111111111111111111111111111111111111111111111111111"
transaction hash: 0x68a6c3177bfae972c578dde9cd7e30e3ace3600f772c6a3f76a47a543c6c3b2c
---------------------------------------------------------------------------------------
transaction status: 0x16
---------------------------------------------------------------------------------------
Transaction inputs:
Input value size:1
Input types: (STRING)
Input values:(0xbbbb11111111111111111111111111111111111111111111111111111111111111111)
---------------------------------------------------------------------------------------
Receipt message: Not authorized to issue
Return message: Not authorized to issue
Return values:null
---------------------------------------------------------------------------------------
```

可见：Receipt message: Not authorized to issue（**未授权，拒绝发证**）。

教务处授权：

loadAccount account/ecdsa/0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24.pem

```
[group:1]> loadAccount account/ecdsa/0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24.pem
Load account account/ecdsa/0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24.pem success!
```

唤醒控制台记忆：

deploy RbacCertificateEvidence.sol

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0x7aed84cdb9ac73532437bfebd8cfc4d500cb1f2c4ef23e38341147df63f37e49
contract address: 0xcb0f8bc136266553a171b2f5b9d4dab32ad8808b
currentAccount: 0x9cfc1cbbe71b74f8446ad1af8d8bb82283df8b24
```

教务处给发放权限：

call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 authorizeIssuer 0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f

```
[group:1]> call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 authorizeIssuer 0x86a3a94c
b34e22de45e6cc3035a12744fc2d6c9f
transaction hash: 0x04bb56ec0e090842dee38b59224970933e6bd3160a286f57e825937c09201765
---------------------------------------------------------------------------------------
transaction status: 0x0
description: transaction executed successfully
---------------------------------------------------------------------------------------
Transaction inputs:
Input value size:1
Input types: (ADDRESS)
Input values:(0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f)
---------------------------------------------------------------------------------------
Receipt message: Success
Return message: Success
Return values:[]
---------------------------------------------------------------------------------------
Event logs
Event: {}
```

可见：transaction status: 0x0 和 Receipt message: Success，这说明**教务处**已经正式给（0x86a3...）下发了**发证权限。**

切换回无权限账户：

loadAccount account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem

```
[group:1]> loadAccount account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem
Load account account/ecdsa/0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f.pem success!
```

唤醒控制台记忆：

deploy RbacCertificateEvidence.sol

```
[group:1]> deploy RbacCertificateEvidence.sol
transaction hash: 0x304761d73333ece00d70dfd5e74fb7ecddfbb8a108ebcdc83df61ba172471def
contract address: 0xd5b28b442ce868d28fdcaa44863a975cd5b1dd0f
currentAccount: 0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f
```

再次执行发证（刚刚失败的那个哈希）：

call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 createEvidence "0xbbbb1111111111111111111111111111111111111111111111111111111111111111"

```
[group:1]> call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 createEvidence "0xbbbb1111
11111111111111111111111111111111111111111111111111111111"
transaction hash: 0x5514443164d40d5f4a1c767a2b1fc8b3d0eabdb6ce774995683c9d61479e2aad
---------------------------------------------------------------------------------------
transaction status: 0x0
description: transaction executed successfully
---------------------------------------------------------------------------------------
Transaction inputs:
Input value size:1
Input types: (STRING)
Input values:(0xbbbb1111111111111111111111111111111111111111111111111111111111111111)
---------------------------------------------------------------------------------------
Receipt message: Success
Return message: Success
Return values:[]
---------------------------------------------------------------------------------------
Event logs
Event: {}
```

可见：transaction status: 0x0 和 Receipt message: Success，这说明刚才那个被拦截的无权限账户，获得教务处授权后，已经成功将数据写入了区块链。

最后一步测试：任何人（HR）都能验证：

依然保持在（现已授权）账户，直接输入：

call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 verifyEvidence "0xbbbb1111111111111111111111111111111111111111111111111111111111111111"

```
[group:1]> call RbacCertificateEvidence 0xe56915632c7ec33b092e72b4749f334da3bef6f8 verifyEvidence "0xbbbb1111
1111111111111111111111111111111111111111111111111111111111111111"
---------------------------------------------------------------------------------------------
Return code: 0
description: transaction executed successfully
Return message: Success
---------------------------------------------------------------------------------------------
Return value size:3
Return types: (ADDRESS, UINT, BOOL)
Return values:(0x86a3a94cb34e22de45e6cc3035a12744fc2d6c9f, 1794144055549, true)
---------------------------------------------------------------------------------------------
```

可见：

0x86a3...：证明这是无权限账户刚发证的。

1794...：精准的时间戳。

true：验证通过，数据不可篡改！